



Name of the module: DECIMATOR
Location in GIT repository: vspa_common\vspa-lib\decimator

Overview:

This module provides kernels to perform 2x, 4x and 8x down sampling (decimation). In order to prevent spectrum aliasing the decimator is preceded by a Low Pass FIR Filter. A naïve implementation would perform useless computation since output samples after filtering are discarded by the elementary decimator. We will use the polyphase filter approach to avoid performing such useless operations. The 4x and 8x decimators will be implemented as cascaded 2x decimators.

Assumptions:

The following assumptions were made:

- Input buffer precision is *Fix Point Half Precision*
- Filter precision is *Floating Point Single Precision*
- Output buffer precision is *Fix Point Half Precision*
- Input/output buffers are complex but filter coefficients are **REAL** corresponding to a Linear Phase FIR filter:
 - design of such a filter can be performed using the '*firls*' function in Matlab
- The filter coefficients are provided to the decimator kernels in an inverted order compared to their natural order:
 - If $h_0, h_1, h_2, \dots, h_{N-1}$ is the filter natural order (yielded by a filter design procedure) these coefficients will be provided in VSPA DMEM in the order $h_{N-1}, h_{N-2}, \dots, h_1, h_0$
 - The convolution operation with the filter coefficients naturally ordered will be translated to a correlation operation with the filter coefficients inversely ordered

Type of code:

- VCPU assembly functions:
 - decimator_2x_32hf
 - decimator_2x_x32hf
 - decimator_4x_32hf
 - decimator_4x_x32hf
 - decimator_8x_32hf
 - decimator_8x_x32hf

Assembly Functions API:

```
void decimator_2x_32hf( cfixed16_t *inp_p,  
                        cfixed16_t *out_p,  
                        cfixed16_t *inp_circ_p,  
                        size_t      inp_circ_size);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
out_p – output pointer for the current block of data



inp_circ_p – the start address of the input circular buffer
inp_circ_size – the size (in half-words) of the input circular buffer

```
void decimator_2x_x32hf( cfixed16_t *inp_p,  
                        cfixed16_t *out_p,  
                        cfixed16_t *inp_circ_p,  
                        size_t      inp_circ_size,  
                        uint32_t     num_out_lines);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
out_p – output pointer for the current block of data
inp_circ_p – the start address of the input circular buffer
inp_circ_size – the size (in half-words) of the input circular buffer
num_out_lines – the number of output lines where each output line consists in 32 complex output samples

```
void decimator_4x_32hf( cfixed16_t      *inp_p,  
                       cfixed16_t      *out_p,  
                       cfixed16_t      *inp_circ_p,  
                       DECIM_4X_PERSIST_MEM_T *persist_p,  
                       size_t           inp_circ_size);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
out_p – output pointer for the current block of data
inp_circ_p – the start address of the input circular buffer
persist_p – pointer to a persistent memory area to hold data between subsequent calls
inp_circ_size – the size (in half-words) of the input circular buffer

```
void decimator_4x_x32hf( cfixed16_t      *inp_p,  
                        cfixed16_t      *out_p,  
                        cfixed16_t      *inp_circ_p,  
                        DECIM_4X_PERSIST_MEM_T *persist_p,  
                        size_t           inp_circ_size,  
                        uint32_t         num_out_lines);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
out_p – output pointer for the current block of data
inp_circ_p – the start address of the input circular buffer
persist_p – pointer to a persistent memory area to hold data between subsequent calls
inp_circ_size – the size (in half-words) of the input circular buffer
num_out_lines – the number of output lines where each output line consists in 32 complex output samples

```
void decimator_8x_32hf( cfixed16_t      *inp_p,  
                       cfixed16_t      *out_p,  
                       cfixed16_t      *inp_circ_p,  
                       DECIM_8X_PERSIST_MEM_T *persist_p,  
                       size_t           inp_circ_size);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
out_p – output pointer for the current block of data
inp_circ_p – the start address of the input circular buffer
persist_p – pointer to a persistent memory area to hold data between subsequent calls



inp_circ_size – the size (in half-words) of the input circular buffer

```
void decimator_8x_x32hf( cfixed16_t      *inp_p,
                        cfixed16_t      *out_p,
                        cfixed16_t      *inp_circ_p,
                        DECIM_8X_PERSIST_MEM_T *persist_p,
                        size_t          inp_circ_size,
                        uint32_t         num_out_lines);
```

where

inp_p – input pointer for the current block of data (within the input circular buffer)
 out_p – output pointer for the current block of data
 inp_circ_p – the start address of the input circular buffer
 persist_p – pointer to a persistent memory area to hold data between subsequent calls
 inp_circ_size – the size (in half-words) of the input circular buffer
 num_out_lines – the number of output lines where each output line consists in 32 complex output samples

Since 4x and 8x decimators are implemented as cascaded 2x decimators, scratch buffers are needed to store the output of the intermediate 2x decimators. Such scratch buffers are provided at link-time as global symbols and not via API. These scratch buffers are allocated statically by the kernel. The integrator can reuse the scratch buffers between subsequent kernel calls.

```
cfixed16_t DECIM_4X_SCRATCH_MEM[DECIM_MAX_OUT_LINES * DECIM_SAMP_PER_LINE * DECIM_FACT_2X];
cfixed16_t DECIM_8X_SCRATCH_MEM[DECIM_MAX_OUT_LINES * DECIM_SAMP_PER_LINE * DECIM_FACT_4X];
```

Also the processing of the intermediate results of the cascaded 2x decimators requires also 1 line of history for each such intermediate buffer (in total 1 line for the 4x decimator and 2 lines for the 8x decimator). The data history of these buffers is transmitted to one kernel call from the previous kernel call. Hence this data will be stored in a persistent type of memory, allocated by the integrator and provided in the function API. The integrator will NOT reuse the persistent buffers and must guarantee that their content remains unchanged between subsequent kernel calls. The following type definitions are defined in the kernel header file for the 4x and 8x decimators and should be used by the integrator for persistent memory allocation.

```
typedef cfixed16_t DECIM_4X_PERSIST_MEM_T[DECIM_SAMP_PER_LINE];
typedef cfixed16_t DECIM_8X_PERSIST_MEM_T[2 * DECIM_SAMP_PER_LINE];
```

Before the first ever kernel call, the persistent storage must be cleared since the convention is that for first ever available input data, its history is all 0's.

The filter coefficients buffer is globally allocated and provided at link-time as a global symbol and not through API. The buffer is defined as:

```
float32_t DECIM_FLT_BUFF[DECIM_MAX_FLT_LEN] __attribute__((aligned(_VSPA_AU_COUNT * 4)));
```

The filter length is a constant macro definition which can be changed by the integrator with the desired value (even value between 16 and 32 inclusively):

```
#define DECIM_FLT_LEN          16
```



The filter length limits are also specified with 2 macros:

```
#define DECIM_MIN_FLT_LEN    16
#define DECIM_MAX_FLT_LEN    32
```

Implementation plan

The implementation is described here for the 2x decimator. We denote with x_n the input samples, with h_n the filter coefficients after their natural order is inverted and with y_n the output samples. The input-output relation for the 2x decimator is:

$$\begin{aligned} y_0 &= h_0 * x_0 + h_1 * x_1 + h_2 * x_2 + h_3 * x_3 + \dots \\ y_1 &= h_0 * x_2 + h_1 * x_3 + h_2 * x_4 + h_3 * x_5 + \dots \\ &\dots \end{aligned}$$

Taking into account that the filter coefficients h_n are real we can rewrite the two relations above separately for the real and imaginary part (denoting with xr_n and xi_n the real and imaginary parts of x_n):

$$\begin{aligned} y_{r0} &= h_0 * xr_0 + h_1 * xr_1 + h_2 * xr_2 + h_3 * xr_3 + \dots \\ y_{i0} &= h_0 * xi_0 + h_1 * xi_1 + h_2 * xi_2 + h_3 * xi_3 + \dots \\ y_{r1} &= h_0 * xr_2 + h_1 * xr_3 + h_2 * xr_4 + h_3 * xr_5 + \dots \\ y_{i1} &= h_0 * xi_2 + h_1 * xi_3 + h_2 * xi_4 + h_3 * xi_5 + \dots \\ &\dots \end{aligned}$$

We will use the VSPA VAU using the following Smodes for the source registers:

- S0mode = S0i1r1i1r1 (repeats two filter coefficients, say h_0 and h_1 , in the entire S0 source register)
- S1mode = S1i2i1r2r1 (loads the input samples with the grouping xi_1, xi_0, xr_1, xr_0 for each 4 elements)
- S2mode = S2zeros

Each cycle operation uses one input line (32 samples) and 2 filter coefficients. Next operation:

- will use the next 2 filter coefficients in S0
- will use the same input line but shifted with 2 complex samples:
 - S1 is shifted each cycle with 4 half-words
- next input line is fetched after the exhaustion of all filter coefficients for current input line
- first iteration of one input line uses **cmad** as operation while for the next **cmac** is used for accumulation with previous results

The next two tables illustrates the VAU source registers for two consecutive cycles:

Iteration #0: cmad

idx	...	7	6	5	4	3	2	1	0
S0	...	h_1	h_0	h_1	h_0	h_1	h_0	h_1	h_0
S1	...	xi_3	xi_2	xr_3	xr_2	xi_1	xi_0	xr_1	xr_0
S2	...	0	0	0	0	0	0	0	0



Iteration #1: cmac

idx	...	7	6	5	4	3	2	1	0
S0	...	h_3	h_2	h_3	h_2	h_3	h_2	h_3	h_2
S1	...	xi_5	xi_4	xr_5	xr_4	xi_3	xi_2	xr_3	xr_2
S2	...	0	0	0	0	0	0	0	0

The pseudocode describing the *cmad* operation (ignoring the pipeline delay) is:

```
for (i=0; i<64; i +=2)
    V[i] = S0[i]* S1[i] + S0[i+1] *S1[i+1] + S2[i];
```

After the exhaustion of all filter coefficients and after the pipeline delay we will obtain in the VAU destination register the desired y_n output corresponding to the first input line.

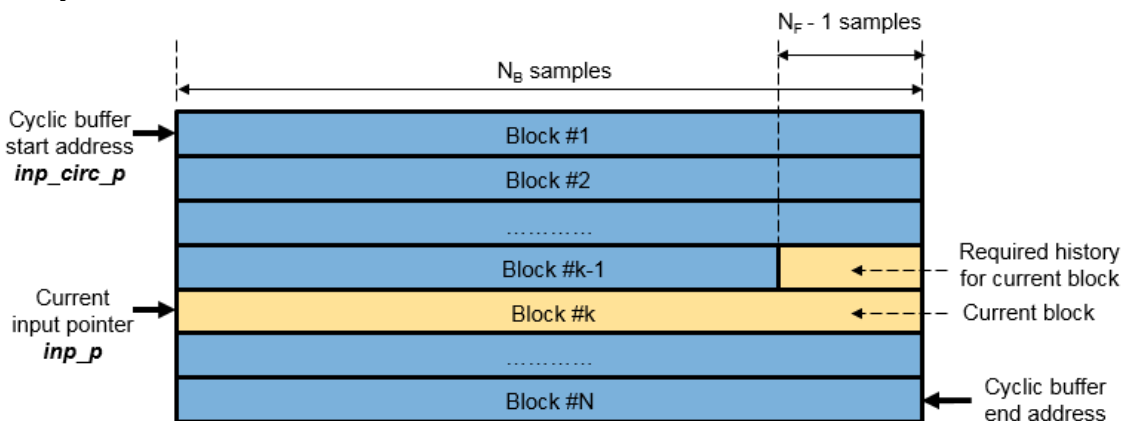
idx	...	7	6	5	4	3	2	1	0
V	...	0	yi_1	0	yr_1	0	yi_0	0	yr_0

Two such outputs (1/2 output line per 1 input line) are coupled for one line transfer to DMEM. The procedure is repeated for all input lines to obtain the entire output.

From the VAU pipeline and loop control perspective, for better efficiency, 2 sets of kernels were implemented: one set for processing 32 complex output samples (the kernels with the “_32hf” suffix) and one set for higher multiples of 32 samples (the kernels with the “_x32hf” suffix).

Memory management

The decimator will operate on subsequent blocks of input data. The block size is chosen according to the desired tradeoff between latency (low latency for smaller blocks) and VSPA AU utilization efficiency (higher efficiency for larger blocks). Since operation on a given block of input data requires the previous block of input data (history) being available, the proper memory management must be performed with a cyclic buffer. The memory management is depicted schematically below:





The cyclic buffer is partitioned in N blocks each N_B samples. Any address manipulation is performed in a cyclically manner inside the cyclic buffer. The cyclic buffer is described by a start address and size (in half-words). If we denote with N_F the number of filter coefficients (filter taps) then processing the current block $\#k$ requires $N_F - 1$ samples from the previous block ($\#k-1$). If the current block being processed is the first ($\#1$) then the history is taken from the last block ($\#N$). The number of blocks is chosen according to the following cases:

- if the input data is **NOT double buffered** then a number of $N=2$ blocks will be used:
 - one block to store the previous input data chunk (used as history)
 - one block for processing current input data chunk
- if the input data is **double buffered** then a number of $N=3$ blocks will be used:
 - one block to store the previous input data chunk (used as history)
 - one block for processing current input data chunk
 - one block for next input data chunk (e.g. fetched in parallel with the current processing via DMA)

The number of input blocks is defined with a macro:

```
#define DECIM_NUM_INP_BUFFERS    2
```

The block size must be greater than the required history size which is $N_F - 1$ samples. The memory management described above applies for the input data of the 2x decimator. Since 4x and 8x decimators are implemented as cascaded 2x decimators, each intermediate data uses similarly the memory management described above but separated in two partitions bases on their usage:

- scratch memory – used to hold the output of an intermediate 2x decimator; this memory region can be reused by the integrator between kernel calls.
- persistent memory – used to hold the output history of an intermediate 2x decimator; this memory region CANNOT be reused by the integrator and must be maintained between kernel calls.

Each kernel will be provided with:

- **inp_p** - the input pointer of the current input data block (the start of the current block within the input circular buffer)
- **inp_circ_p** - the start address of the whole input circular buffer
- **inp_circ_size** – the size (in half-words) of the whole input circular buffer

The manipulation of the intermediate results (inside scratch/persistent buffers) will be performed inside the kernel, hidden to the integrator.

Since the convention is that the history of first ever available input data is all '0's the integrator must assure that before FIRST EVER kernel call the following buffers are all '0's:

- the history portion of the input circular buffer (last $N_F - 1$ samples of input data)
- the persistent memory buffers corresponding to the history of the intermediate output buffers (for 4x and 8x decimators)



The maximum number of output lines is defined as a macro and used for input, output and scratch buffer memory allocation. The integrator can change the macro to the desired value:

```
#define DECIM_MAX_OUT_LINES 4
```

Limitations

The approach described above has the following limitations:

- The **filter length must be even** since filter coefficients are used in pairs each cycle
 - effectively the filter can be odd but must be padded with a 0 and used as an even filter
- The supported number of filter coefficients is 16,18,...,32
- The output data size is restricted to a corresponding **integer number of output lines** which means a multiple of 32 complex half-fixed output samples

DMEM requirements:

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
DECIM_FLT_BUFF	64	64	Kernel

The DMEM minimum requirements (separately for each kernel, for processing 1 output line per call) are:

Decimator 2x

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
input buffer	2 x 128	64	Caller
output buffer	64	64	Caller

Decimator 4x

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
input buffer	2 x 256	64	Caller
output buffer	64	64	Caller
DECIM_4X_SCRATCH_MEM	128	64	Kernel
DECIM_4X_PERSIST_MEM	64	64	Caller

Decimator 8x

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
input buffer	2 x 512	64	Caller
output buffer	64	64	Caller
DECIM_8X_SCRATCH_MEM	256	64	Kernel
DECIM_8X_PERSIST_MEM	128	64	Caller

Test plan:



- **Feature that are going to be tested:**
 - Different decimation factors (2x, 4x, 8x)
 - Different filter lengths (all even lengths between 16 and 32)
 - Different output sizes (1, 2,3,4 lines) which corresponds to specific input block sizes
 - Different number of blocks to test block-wise processing (circular buffer and history management)
- Take a look in **Decimator_Testplan.xlsx** for test plan details